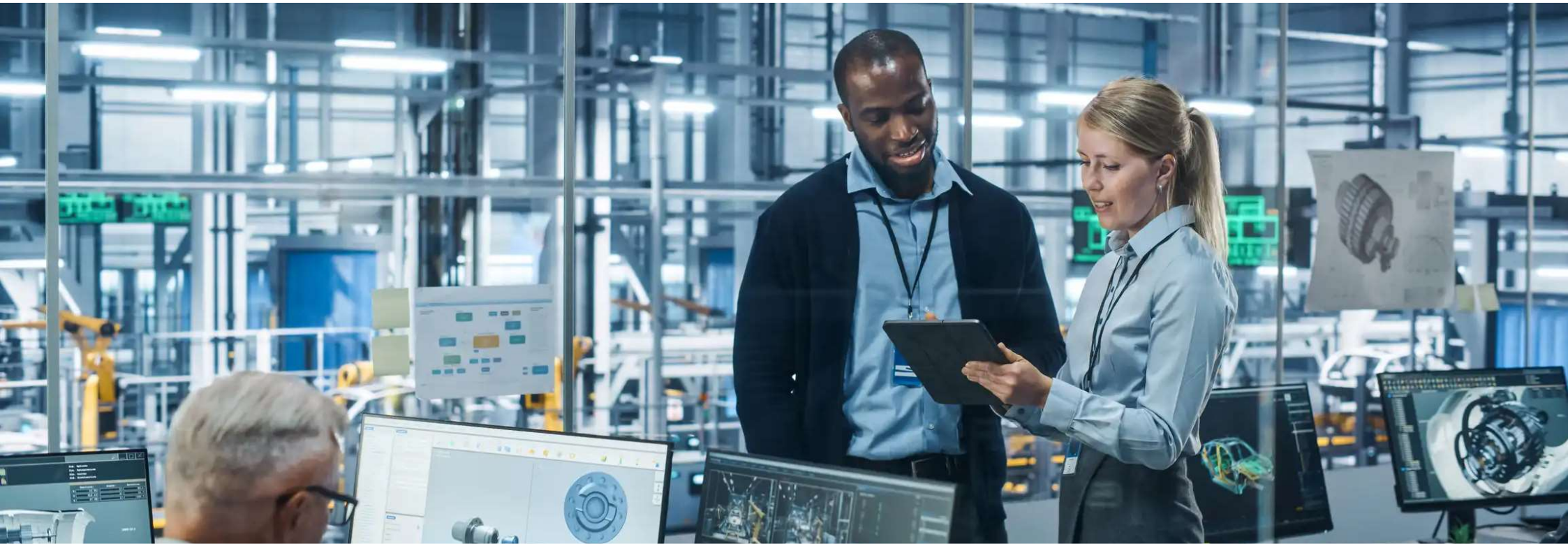


PERIODIC AND REAL-TIME DISPATCHING RULE SELECTION FOR THE DYNAMIC JOB SHOP SCHEDULING PROBLEM

Nuno Marques, Gonçalo Figueira, Luís Guimarães

INESC TEC & Faculdade de Engenharia, Universidade do Porto

33th EURO Conference

30th June to 3rd July 2024, Copenhagen

Financing: *This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 952060.*

Disclaimer: *The sole responsibility for the contact lies with the authors. It does not necessarily reflect the opinion of the CNECT or the European Commission (EC). CNECT or the EC are not responsible for any use that may be made of the information contained therein.*

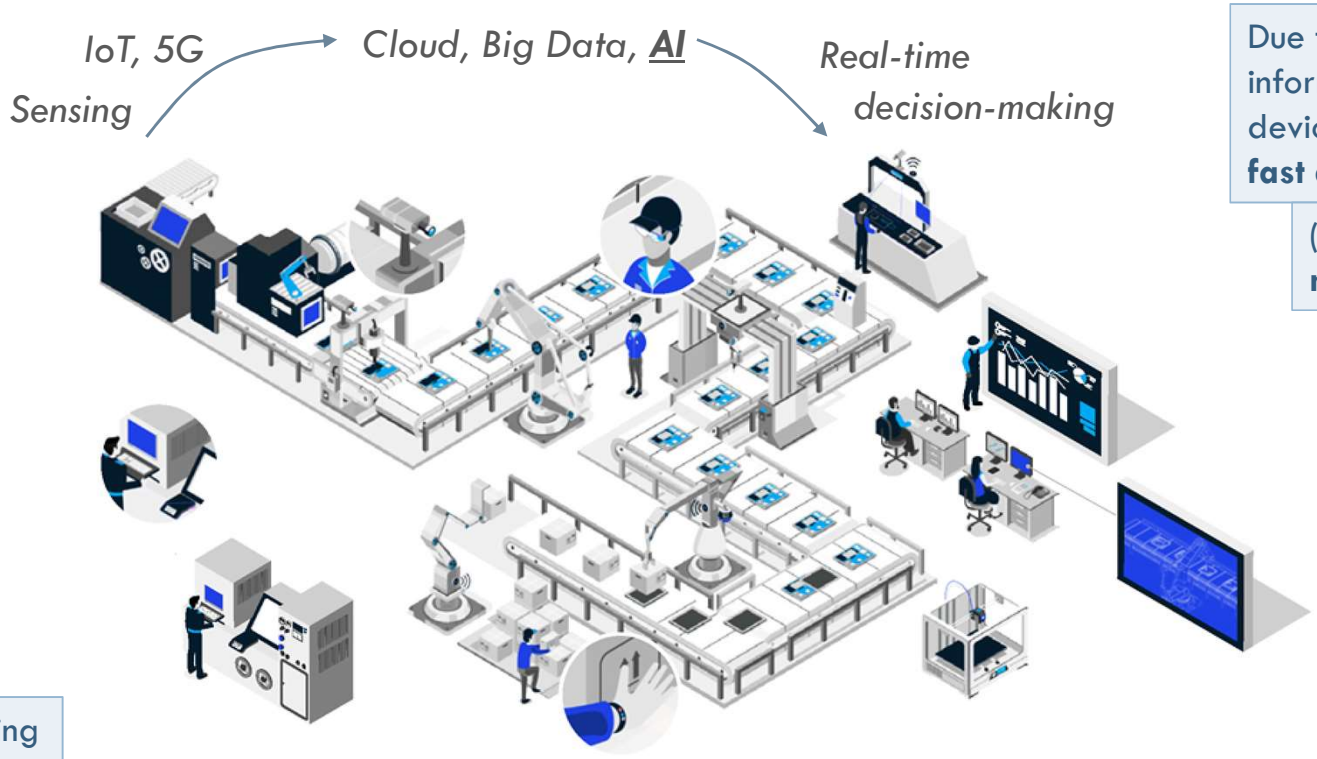
INDUSTRY 4.0 ENABLES REAL-TIME SCHEDULING

Industry 4.0 growth prediction: **15% CAGR** until 2030 (UBS, 2024)

By 2030, **25 zettabytes** of data will require real-time processing. (IDC, 2019)

HENKEL collects **1 million datapoints** per day (Etteplan)

Phillips razor manufacturing facility requires a staff of **9 people**. (Marr, 2023)



Agrawal, M. et al. (2020). *Industry 4.0: Reimagining manufacturing operations after COVID-19*. McKinsey & Company.

Due to the large amount of information generated daily from IoT devices, **real-time scheduling** and **fast data processing** are essential.

(…) Responses must be in **milliseconds** at times.

Scheduling algorithms, one of the **most important issues** of real-time systems,…

(Khalej et al., 2023)

This situation (online data collection) poses a major challenge to the current scheduling system because **dynamic changes** in the shop floor require **real-time responses.** (Zhou et al., 2019)

DRS PERFORM UNDER UNCERTAINTY, BUT THERE IS STILL ROOM FOR IMPROVEMENT

DRs examples:

- SPT: shortest processing time;
- FIFO: first-in-first-out.

DRs became popular in dynamic scheduling due to…

- Speed of computation; (Golmakani and Birjandi, 2013)
- Performance under uncertainty. (Rajabinasab and Mansour, 2011)

However, there is no universal DR (Baker, 1984)…

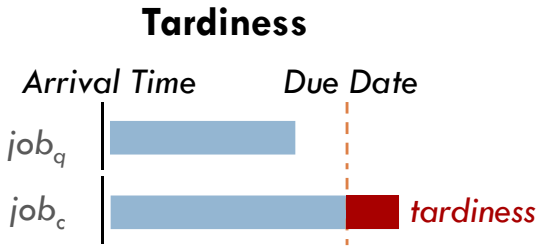
- Optimization criteria, e.g.: flowtime vs tardiness;
- Operating conditions., e.g.: tight vs loose load conditions.

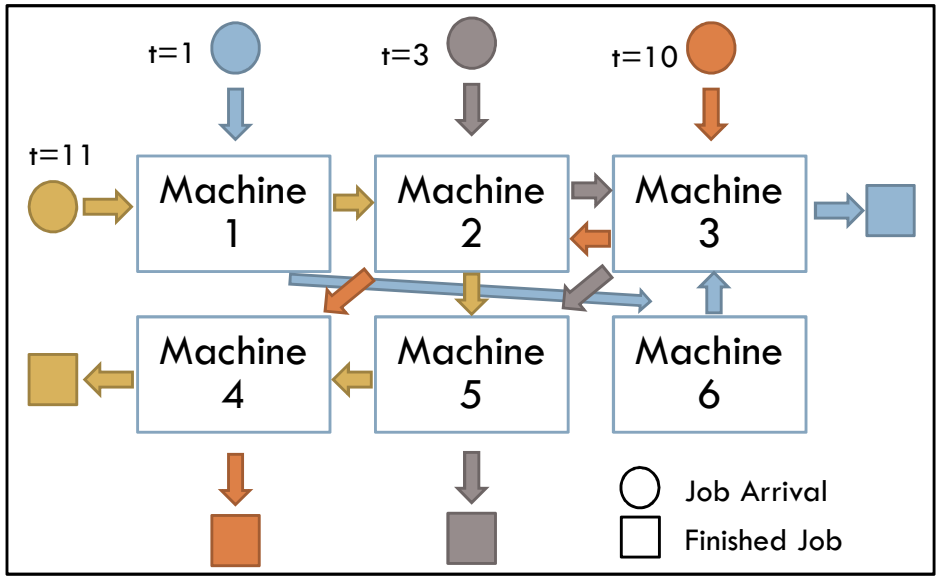
DRSS – **D**ispatching **R**ule **S**election **S**ystem
Dynamically select DRs as conditions change

THE DYNAMIC JOB SHOP

The DJSSP:

- Jobs arrive randomly;
- Each job has a route of operations;
- Each operation is to be executed in a machine;
- The optimization criterion in focus is tardiness:





DRs are responsible for sorting queues.

Time

EDD (Jackson 1955)
SPT (Smith 1956)

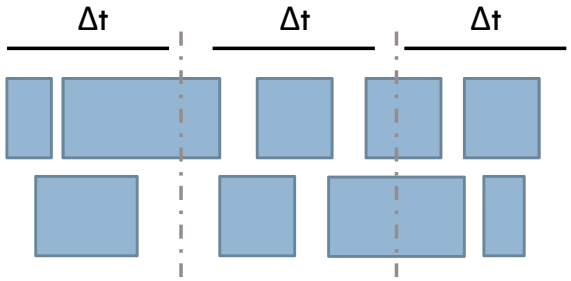
$PT \cdot e^{v} + PT \cdot S / RPT \cdot e^{-v} + WINQ$
(Raghu and Rajendran 1993)

$2PT + PT \cdot S^{+} / RPT + WINQ$
(Ferreira et al. 2022)

DR SELECTION APPROACHES

PRS

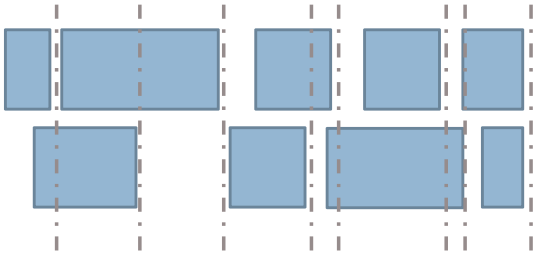
Periodic Rule Selection



- Rules stay active for a pre-defined period of time
- Supervised Learning
- Find the best period length to train the models
- Literature: Shiue et al. (2012); Priore et al. (2018); Đurasević and Jakobović (2022)

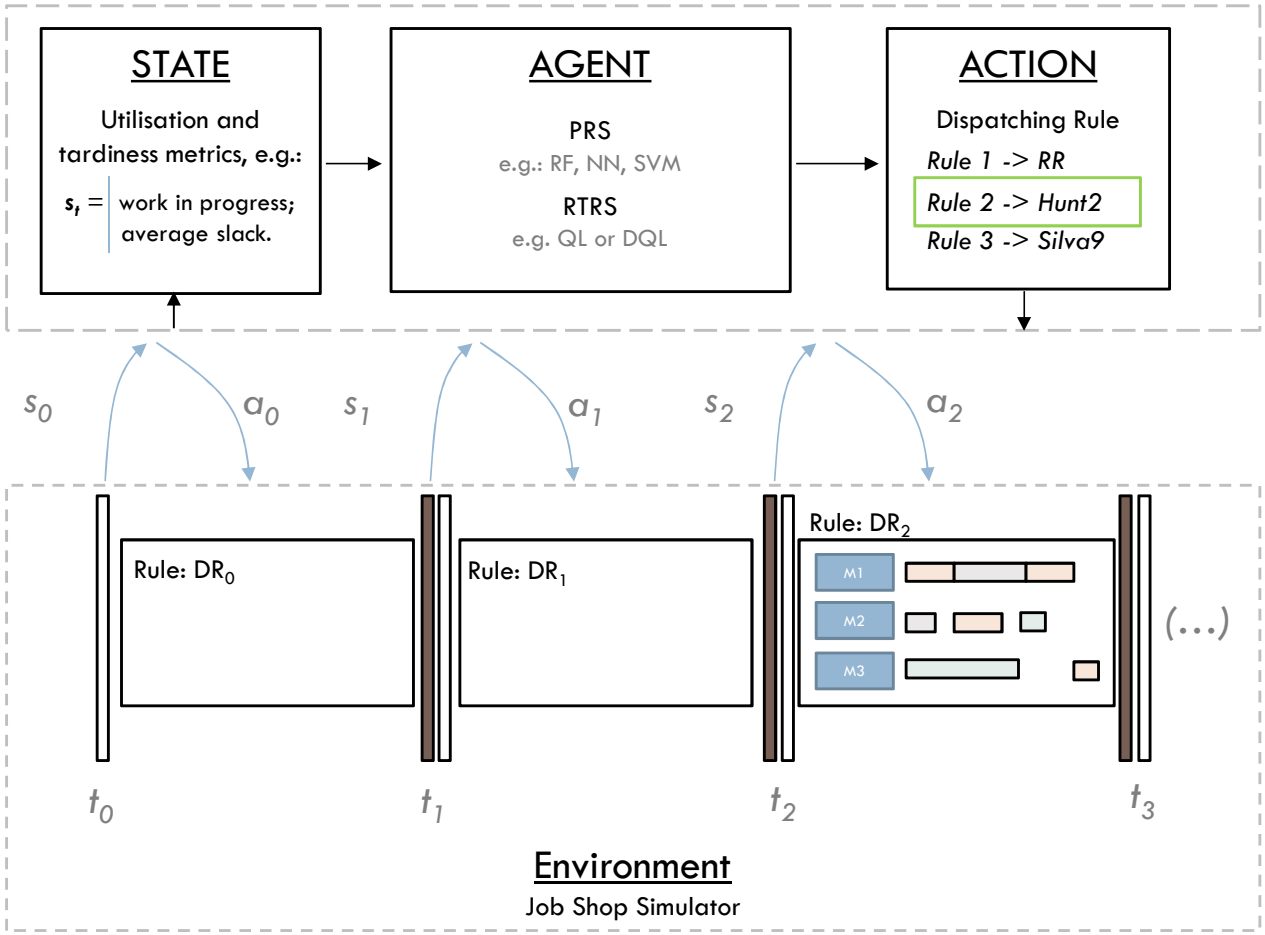
RTRS

Real-Time Rule Selection



- Every time dispatching is necessary the DR is selected
- Reinforcement Learning
- Temporal credit assignment
- Literature: Luo (2020); Wang (2020)

DYNAMIC DR SELECTION SYSTEM

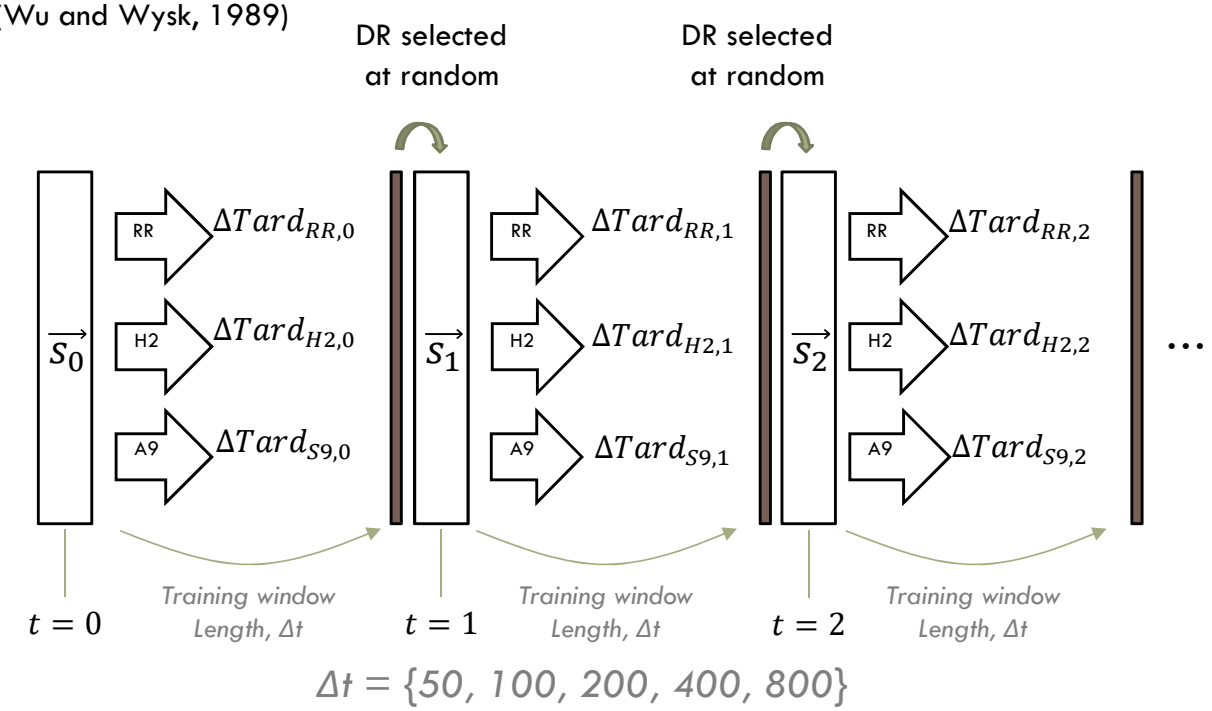


AGENTS FOR PERIODIC RULE SELECTION

How to generate a dataset to train the supervised learning models (RFs, NNs, SVMs)?

Multipass simulation technique:

(Wu and Wysk, 1989)



$\Delta t = \{50, 100, 200, 400, 800\}$

Dataset example:

Regression:

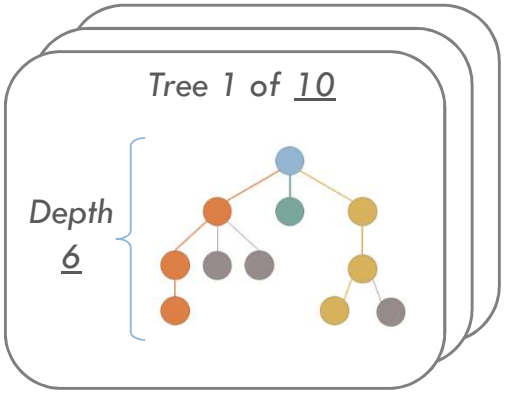
			$\Delta Tard_{RR,t}$	$\Delta Tard_{H2,t}$	$\Delta Tard_{S9,t}$
t=0	$\overrightarrow{S_0}$	S9	5	10	0
t=1	$\overrightarrow{S_1}$	RR	0	7	2
t=2	$\overrightarrow{S_2}$	RR	0	8	3
...	...	...	...	...	...

Classification:

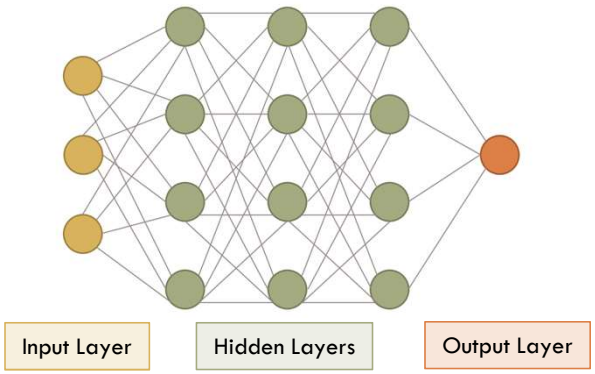
$$\underset{DR}{argmin}\,(\Delta Tard_{DR,t})$$

AGENTS FOR PERIODIC RULE SELECTION

RANDOM FOREST

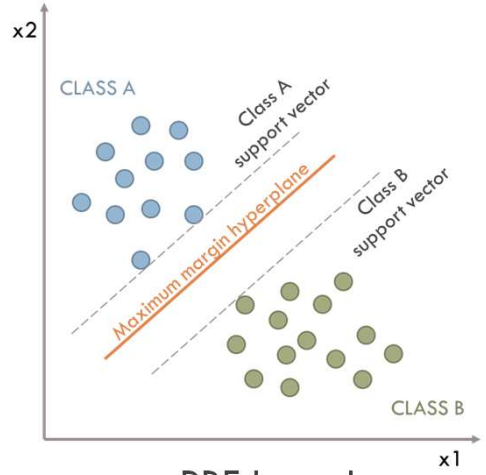


NEURAL NETWORK



Input layer: 2 or 3 neurons
3 hidden layers: 4 neurons
Output layer: 1 neuron
Sigmoid (class.) and ReLu (regr.)

SUPPORT VECTOR MACHINE



RBF kernel

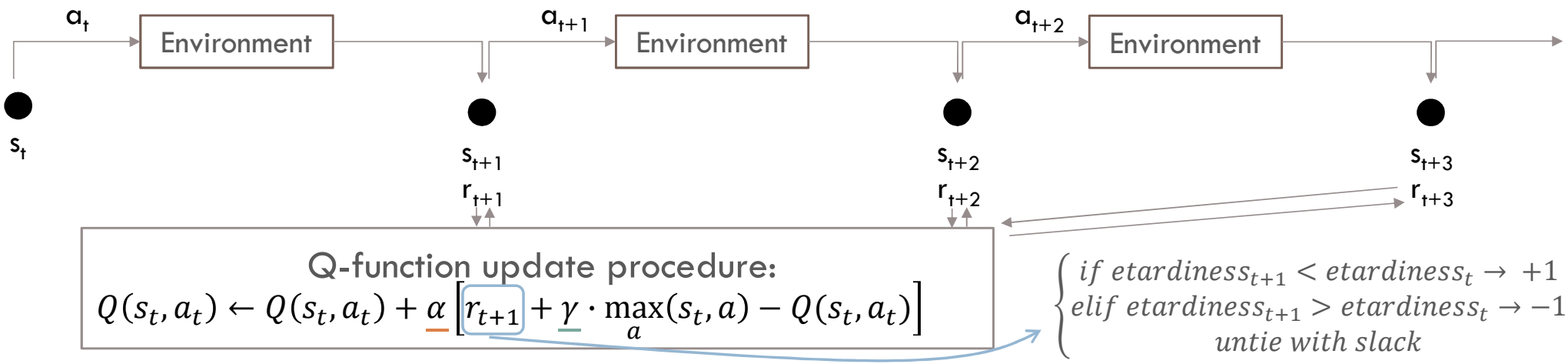
Considering 3 algorithms, 2 learning tasks, 5 training periods, 3 state sets, and 5 seeds → **330 PRS models**

RF, NN, SVM	Class. & Regr.	50, 100, 200 & 400, 800	SS_1, SS_2 & SS_3

AGENTS FOR REAL-TIME RULE SELECTION

RL agents learn by interacting with the environment and by collecting rewards.

Q-Learning in particular learns by updating its policy (the Q-function) at each decision point.



At each *t*, the Q-function is responsible for estimating the expected future rewards of different state action $Q(s_t,a_t)$ pairs.

AGENTS FOR REAL-TIME RULE SELECTION

QL

Q-function takes the form of a look-up table;

State-space must be discretised;

state \ action	a_0	a_1	…
s_0	$Q(s_0,a_0)$	$Q(s_0,a_1)$	…
s_1	$Q(s_1,a_0)$	$Q(s_1,a_1)$	…
…	…	…	…

Table size:

- 2 state vars: 10 x 10 x 3 DRs
- 3 state vars: 10 x 10 x 10 x 3 DRs

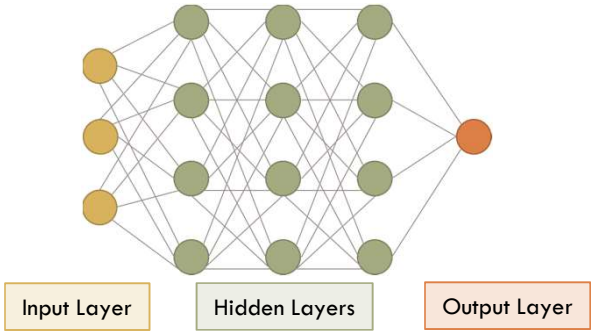
QL update parameters:

- α (learning rate): 0.2, 0.3, 0.4
- γ (discount factor): 0.7, 0.8, 0.9
- ε (exploration rate): 0.9 to 0.1 (linear decay)

DQL is suitable for continuous state spaces

DQL

Q-function takes the form of a neural network;



Network size:

- 2 state vars: 4 x 4x4x4 x 1
- 3 state vars: 5 x 5x5x5 x 1

QL update parameters:

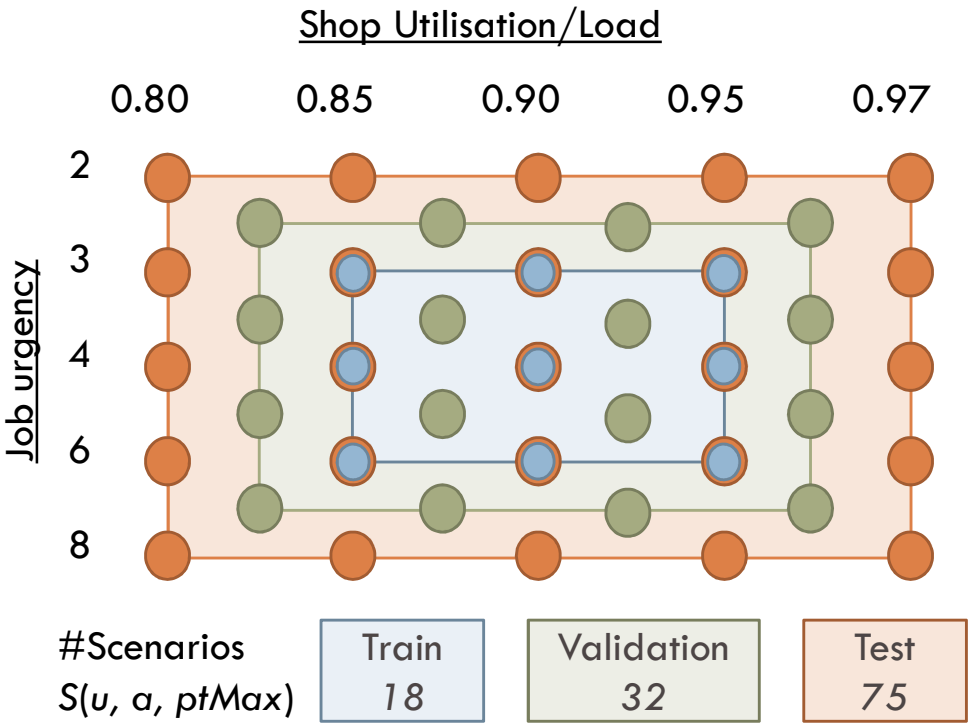
- α (learning rate): 0.001, 0.005, 0.01
- γ (discount factor): 0.80 , 0.85 , 0.90
- ε (exploration rate): 0.9 to 0.1 (linear decay)

Considering 2 algorithms, 3 learning rates, 3 discount factors, 3 state sets, and 5 seeds → **270 runs**

QL & DQL α1, α2 & α3 γ1, γ2 & γ3 SS_1, SS_2 & SS_3

EXPERIMENTS DESIGN

A wide array of **load** and **urgency** scenarios were tested to assess model robustness.



Parameter	Value
#Machines, *M*	20
#Jobs, *J*	2500
#Operations/job, O_i	*U*(2,14)
Processing times	*U*(1,*ptMax*): *ptMax* = {50, 100, 200**} ** only for testing

Tardiness score, *NT*:

$$NT = \left(\prod_s NT_s\right)^{\frac{1}{|s|}}, \quad NT_s = \frac{\text{Model performance in } s}{\text{Reference value for } s}$$

Performance of the best DR

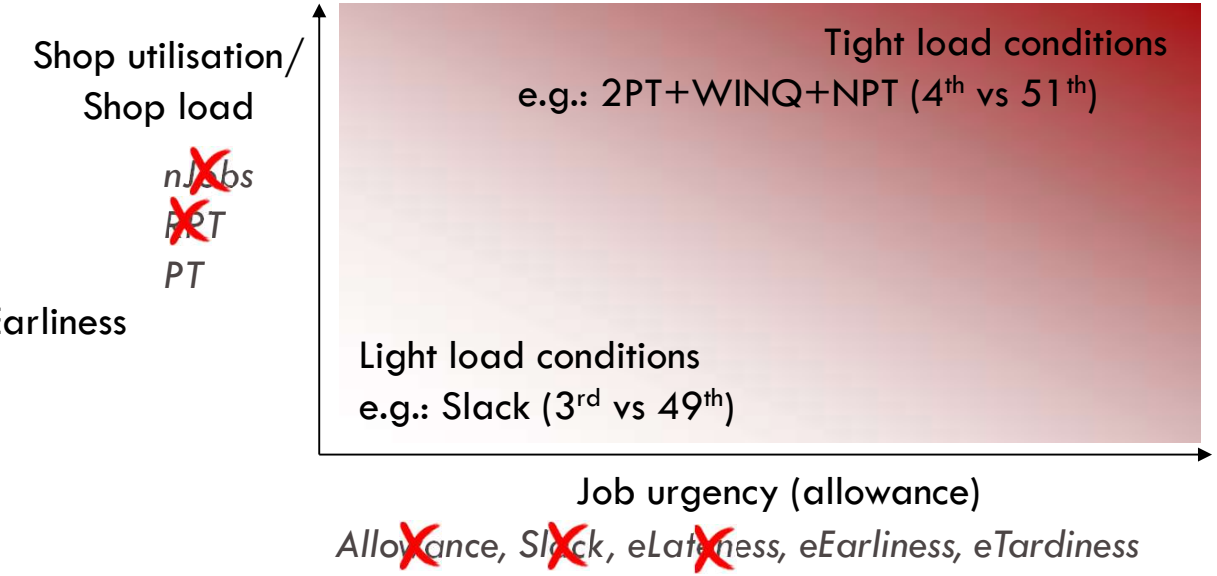
RESULTS - STATE SET

DR performance varies mainly in terms of job shop utilization and job urgency

Several features have been proposed over the years, although many are highly correlated

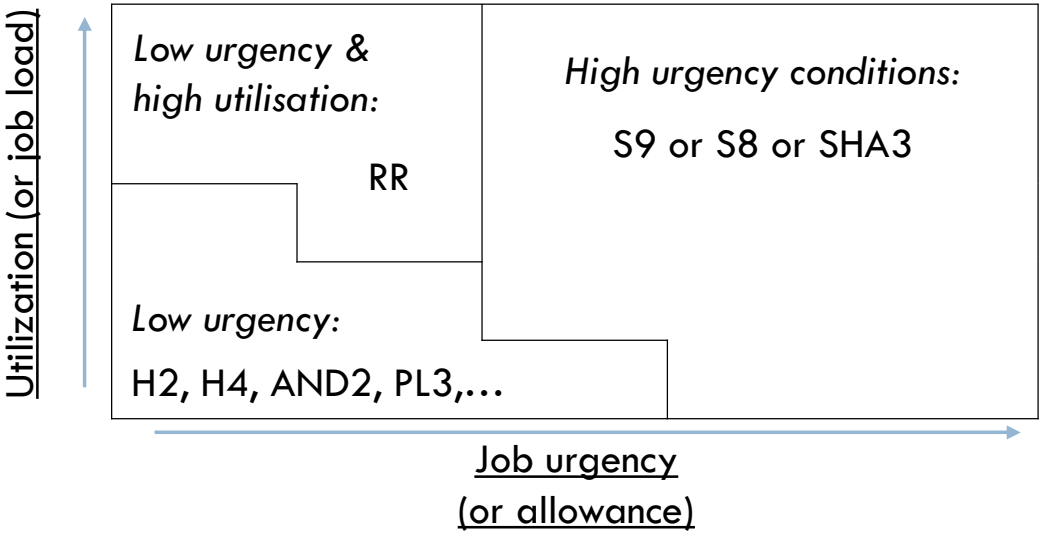
State sets

- SS_1: Processing Times x eTardiness
- SS_2: Processing Times x eEarliness
- SS_3: Processing Times x eTardiness x eEarliness



RESULTS - ACTION SET

After testing DRs from several sources, a complementary set of DRs was found.



Rule 1	Rule 2	Rule 3	Performance
S9	RR	H2	1.036
S9	RR	H4	1.039
S9	RR	AND2	1.039
S9	RR	H1	1.055
S9	RR	OZ1	1.064
SHA3	RR	H2	1.065
SHA3	RR	H4	1.066
…	…	…	…

RESULTS - PRS

Periodic Rule Selection, Supervised Learning

- Larger splits for training
- Shorter lengths in for decision periods

Best Performing Model

Train Data \ Decision Frequency	w50	w100	w200	w400	w800
w50	1.002				
w100	1.010	1.020			
w200	1.000	1.010	1.021		
w400	0.992	1.001	1.012	1.019	
w800	0.992	0.998	1.005	1.009	1.032

(*) tardiness score, validation data

RESULTS - PRS & RTRS

Algo.	Task	Train window length	DR selection frequency	State set	Tardiness score			
					Validation		Test	
					average	wcs	average	wcs
DR	RR	-	-	-	1.146	-	1.219	-
	H2	-	-	-	1.624	-	1.509	-
	A9	-	-	-	1.102	-	1.213	-
RF	Class.	800	50	SS_1	0.995	1.002	1.060	1.065
	Reg.	800	50	SS_3	0.998	1.005	1.000	1.005
NN	Class.	800	50	SS_1	1.013	1.020	1.013	1.020
	Reg.	100	50	SS_1	1.023	1.054	1.039	1.086
SVM	Class.	400	50	SS_3	0.992	-	0.997	-
	Reg.	800	50	SS_3	0.992	-	0.995	-
RL	QL	-	-	SS_1	1.007	1.014	1.060	1.091
	DQL	-	-	SS_3	0.981	0.984	1.034	1.068

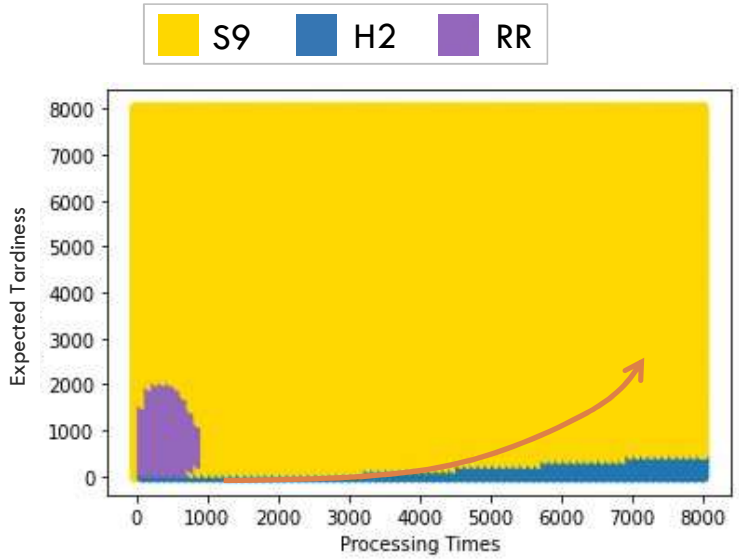
Low performance in validation and test

Good validation & test performance

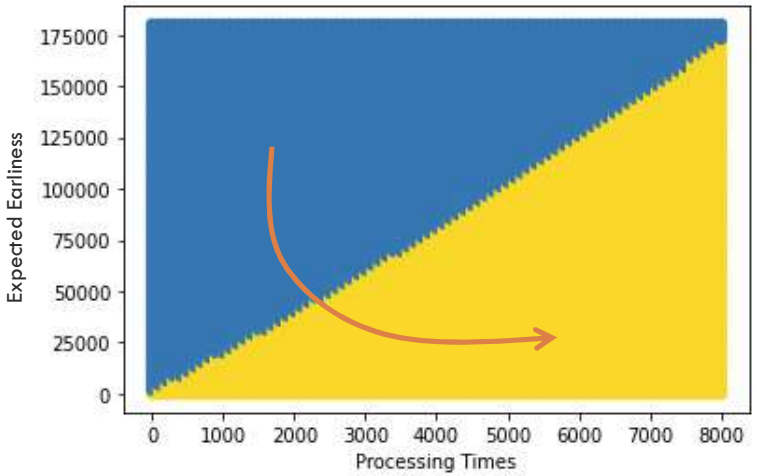
Good validation performance, but bad generalisation

MODEL BEHAVIOUR

Switching between H2 and S9 allows models to beat single DRs



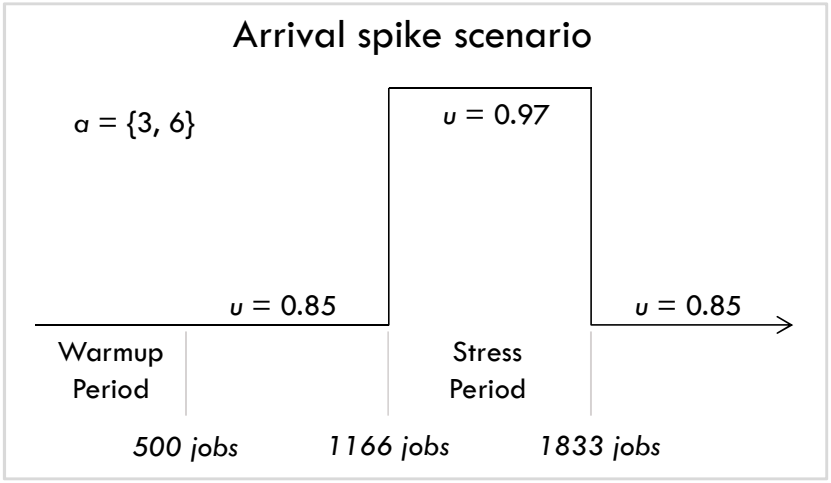
SVM Regression

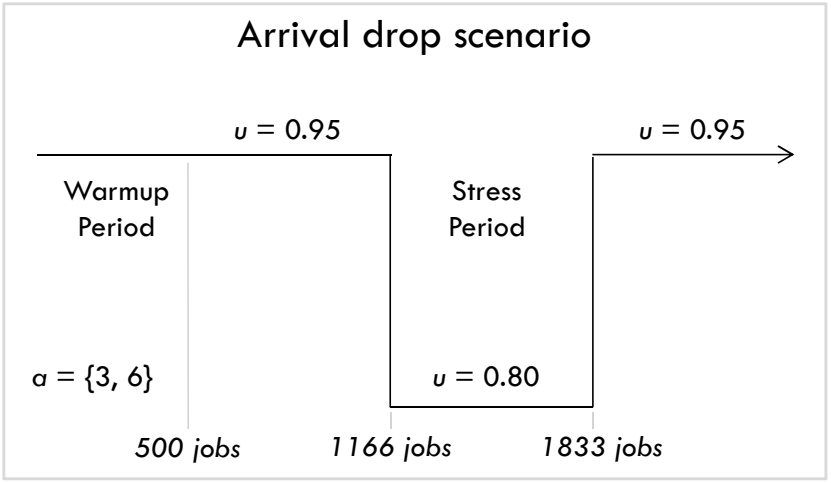


SVM Classification

PERFORMANCE UNDER STRESS CONDITIONS

Instances were modified to assess model behavior under stress conditions.





RESULTS, NON-STATIONARY INSTANCES

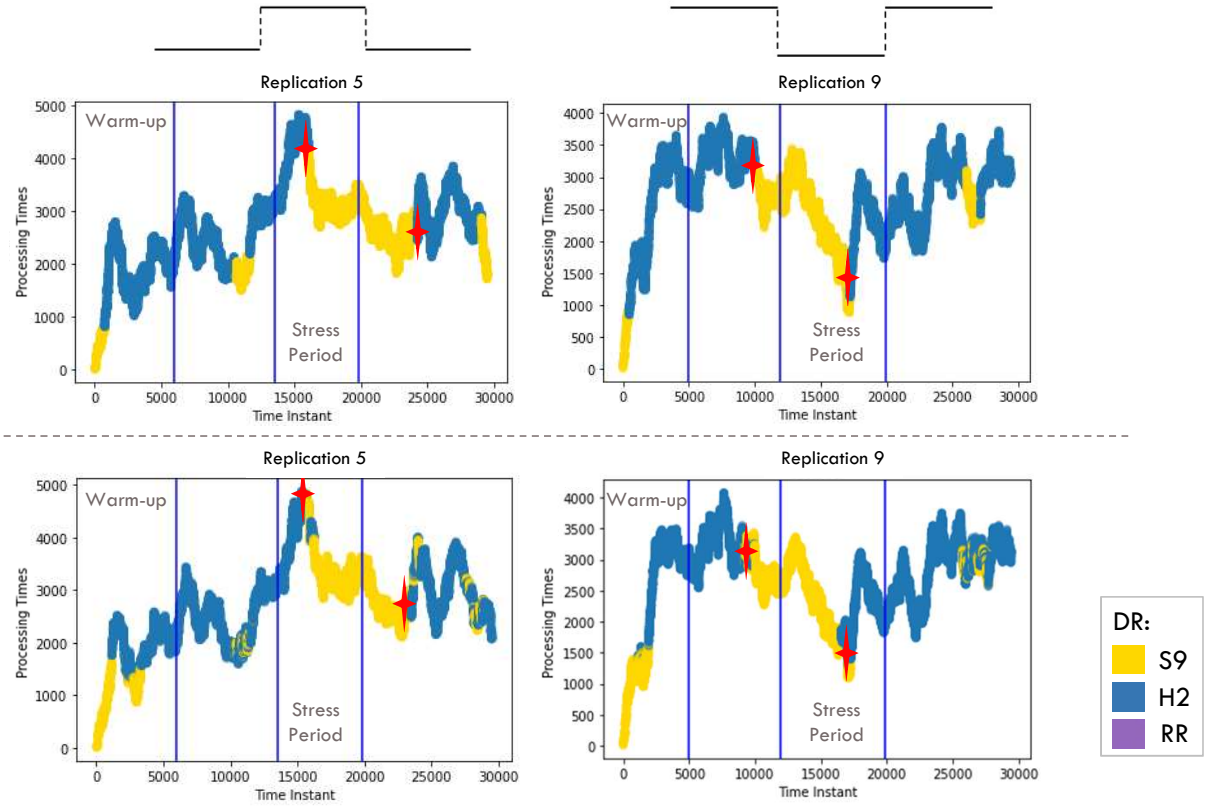
Algo.	Task	Arrival Spike				Arrival Drop				Average	
		Low urgency		Urgent jobs		Low urgency		Urgent jobs			
		avg.	wcs	avg.	wcs	avg.	wcs	avg.	wcs	avg.	wcs
DR	RR	1.000	-	1.117	-	1.000	-	1.138	-	1.062	-
	H2	1.246	-	1.909	-	1.215	-	1.972	-	1.545	-
	A9	1.041	-	1.000	-	1.049	-	1.000	-	1.022	-
RF	Class.	1.004	1.014	1.004	1.006	0.999	1.001	0.995	0.996	1.000	1.004
	Reg.	0.999	1.002	1.000	1.001	0.975	0.980	0.995	0.997	0.992	0.994
NN	Class.	1.000	1.000	1.000	1.001	0.977	0.978	0.994	0.995	0.993	0.993
	! Reg.	1.020	1.062	1.034	1.064	0.991	1.012	1.012	1.028	1.014	1.036
SVM	Class.	1.006	-	0.998	-	0.973	-	0.994	-	0.993	-
	Reg.	1.008	-	0.998	-	0.974	-	0.991	-	0.993	-
RL	! QL	0.980	1.010	1.019	1.032	0.961	1.012	1.012	1.024	0.993	1.007
	DQL	0.989	0.995	1.002	1.010	0.967	0.993	0.993	0.998	0.988	0.991

MODEL BEHAVIOUR, NON-STATIONARY INSTANCES

- RL leads to much higher number of rule changes

PRS

RTRS



FINAL REMARKS

Both PRS and RTRS systems can greatly increase performance over using a single DR.

PRS outperformed RTRS in **stationary** instances.

However, **RTRS** was the best approach for **non-stationary** instances.

For future work…

- Can scheduling performance be improved by agents that schedule jobs directly instead of choosing DRs?